

Keycloak Realm Configuration & Adding New IdPs

This document covers how the Keycloak instance at `sso.aula.de` is set up and how to add new Identity Providers.

Infrastructure

Keycloak runs at `https://sso.aula.de/auth` as a Docker service alongside the aula backend, proxied by nginx.

docker-compose snippet (/opt/keycloak/docker-compose.yml)

```
name: keycloak

networks:
  aula_external:
    external: true
    name: aula_aula_external
  keycloak_internal:
    driver: bridge

services:
  keycloak-db:
    image: postgres:16
    restart: unless-stopped
    environment:
      POSTGRES_DB: keycloak
      POSTGRES_USER: keycloak
      POSTGRES_PASSWORD: CHANGE_ME
    volumes:
      - /opt/keycloak/db:/var/lib/postgresql/data
    networks:
      - keycloak_internal

  keycloak:
    image: quay.io/keycloak/keycloak:26.1
    restart: unless-stopped
    command: start
    environment:
      KC_DB: postgres
      KC_DB_URL: jdbc:postgresql://keycloak-db:5432/keycloak
      KC_DB_USERNAME: keycloak
      KC_DB_PASSWORD: CHANGE_ME
      KC_HOSTNAME: sso.aula.de
```

```
KC_HTTP_RELATIVE_PATH: /auth
KC_HTTP_ENABLED: "true"
KC_PROXY_HEADERS: xforwarded
KC_HOSTNAME_STRICT: "false"
KEYCLOAK_ADMIN: admin
KEYCLOAK_ADMIN_PASSWORD: CHANGE_ME_ADMIN
depends_on:
  - keycloak-db
networks:
  - aula_external
  - keycloak_internal
```

Nginx reverse proxy (inside sso.aula.de server block)

```
location ^~ /auth/ {
    proxy_pass http://keycloak:8080;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_buffer_size 128k;
    proxy_buffers 4 256k;
}
```

Realm Layout

Realm	Purpose
master	Keycloak admin only — do not use for aula
aula	Production realm — all tenants share this
mock-iserv	Simulated IServ IdP for development/testing

Realm Configuration

Admin account

The permanent admin user is aula-admin with the admin role in the master realm.

Admin console: <https://sso.aula.de/auth/admin>

aula realm settings

- **General → Frontend URL:** <https://sso.aula.de/auth> (must be set to avoid issuer mismatches)
- **Login:** User registration OFF, Forgot password OFF (all users come from IdPs)
- **Sessions:** configure SSO session idle/max as needed

aula-backend client (inside aula realm)

Setting	Value
Client ID	aula-backend
Client type	OpenID Connect
Flow	Standard flow only
Valid redirect URIs	https://sso.aula.de/api/v2/auth/sso/callback
Web origins	https://sso.aula.de

Grab the **Client Secret** from the Credentials tab and put it in .env:

```
KEYCLOAK_BASE_URL=https://sso.aula.de/auth
KEYCLOAK_REALM=aula
KEYCLOAK_CLIENT_ID=aula-backend
KEYCLOAK_CLIENT_SECRET=<paste from Keycloak>
KEYCLOAK_REDIRECT_URI=https://sso.aula.de/api/v2/auth/sso/callback
APP_FRONTEND_URL=https://sso.aula.de
```

Mock IServ (development IdP)

mock-iserv realm setup

1. Create realm named mock-iserv
2. **Realm settings → General → Frontend URL:** <https://sso.aula.de/auth>
3. Add test users (teacher1, student1) via **Users → Add user → Credentials** (set non-temporary password)

Broker client in mock-iserv

1. **Clients → Create client**
 - Client ID: keycloak-broker
 - Client type: OpenID Connect
 - Enable Standard flow + Client authentication
2. Valid redirect URI: <https://sso.aula.de/auth/realms/aula/broker/mock-iserv/endpoint>

3. Note the **Client Secret** from the Credentials tab

Add mock-iserv as IdP in the aula realm

1. Switch to aula realm → **Identity Providers** → **Add provider** → **OpenID Connect v1.0**
2. Fill in:

Field	Value
Alias	mock-iserv
Discovery endpoint	https://sso.aula.de/auth/realms/mock-iserv/.well-known/openid-configuration
Client ID	keycloak-broker
Client Secret	(from broker client above)

3. Click **Import** to auto-fill endpoints
4. Set **back-channel URL overrides** to use internal URLs (avoids hairpin NAT issues):

Override	Value
Token URL	http://keycloak:8080/auth/realms/mock-iserv/protocol/openid-connect/token
Logout URL	http://keycloak:8080/auth/realms/mock-iserv/protocol/openid-connect/logout
JWKS URL	http://keycloak:8080/auth/realms/mock-iserv/protocol/openid-connect/certs
User Info URL	http://keycloak:8080/auth/realms/mock-iserv/protocol/openid-connect/userinfo
Authorization URL	https://sso.aula.de/auth/realms/mock-iserv/protocol/openid-connect/auth (keep HTTPS)

5. Save

Test the chain

Open a private browser window and visit:

https://sso.aula.de/auth/realms/aula/protocol/openid-connect/auth?client_id=aula-backend&response_type=code&redirect_uri=https://sso.iserv

Should redirect to the mock-iserv login page.

Adding a New Real IdP (e.g. VIDIS, IServ)

1. Switch to the aula realm → **Identity Providers** → **Add provider** → **OpenID Connect v1.0**
2. Fill in the alias (e.g. vidis or iserv), the discovery endpoint, client ID and secret from the IdP registration
3. Click **Import** to auto-fill endpoints
4. If the IdP is external (not self-hosted on the same server), skip the back-channel overrides
5. Save
6. Enable SSO on the tenant and set sso_provider to the alias:

```
docker compose exec aula-backend.v2 php artisan tinker --execute="
\App\Models\Tenant::where('instance_code', 'TENANT_CODE')
->update(['sso_enabled' => true, 'sso_provider' => 'vidis']);
"
```

VIDIS (when credentials arrive)

Field	Value
Alias	vidis
Discovery endpoint	https://aai-test.vidis.schule/auth/realms/vidis/.well-known/openid-configuration
Client ID	from VIDIS registration
Client Secret	from VIDIS registration
Scopes	openid profile email

SSO Logout Setup

The logout flow destroys both the aula realm session and the upstream IdP session via OIDC RP-initiated logout:

Frontend → IdP logout → aula realm logout → Frontend

The `end_session_endpoint` is discovered automatically from `{iss}/.well-known/openid-configuration`, so it works with any OIDC-compliant provider without hardcoded URLs. No code changes are needed when adding new IdPs.

Keycloak — aula realm (one-time setup)

1. Include realm roles in access tokens

Clients → **aula-backend** → **Client Scopes** — ensure roles is under **Default client scopes**.

2. Allow all users to read their IdP tokens

Realm Settings → **User Registration** → **Default Roles** — add the **broker** realm role.

This lets the backend call `/realms/aula/broker/{provider}/token` after login to retrieve the upstream IdP's `id_token`, which is stored on the user and used as `id_token_hint` during logout.

Per identity provider (repeat for each: mock-iserv, iServ, VIDIS, etc.)

In the aula realm — Identity Provider settings

Identity Providers → `{provider}` → **Settings**

Setting	Value
Store Tokens	ON
Stored Tokens Readable	ON

Without these, Keycloak returns "does not support this operation" from the broker API.

In the IdP's broker client — post-logout redirect URI

For IdPs that are themselves Keycloak realms (mock-iserv, iServ via Keycloak), add to the `keycloak-broker` client:

{IdP realm} → Clients → keycloak-broker → Settings → Valid post logout redirect URIs

`https://sso.aula.de/*`

Without this, Keycloak rejects the `post_logout_redirect_uri` with "Invalid redirect uri".

For non-Keycloak providers (standalone iServ, VIDIS), register `https://sso.aula.de/*` as an allowed post-logout redirect URI in that provider's client config.

How the full logout chain works

```
POST /api/v2/auth/sso/logout
├─ [server] revoke aula realm refresh token
├─ [server] fetch IdP id_token via broker API
└─ returns logout_url:
    {IdP_end_session_endpoint}
    ?id_token_hint={idp_id_token}
    &post_logout_redirect_uri=
    {aula_realm_logout}
    ?id_token_hint={aula_id_token}
    &post_logout_redirect_uri={frontend_url}
```

Browser navigates to `logout_url`:

1. IdP session destroyed
2. aula realm session destroyed
3. User lands on frontend login page

Troubleshooting

Issuer mismatch — make sure Frontend URL is set on every realm to `https://sso.aula.de/auth`.

Back-channel failures (hairpin NAT) — set internal `http://keycloak:8080/...` URLs in back-channel overrides, keep `https://sso.aula.de/...` for browser-facing URLs.

User still logged in after logout — Keycloak session persists. Use a private window, or log out directly at `https://sso.aula.de/auth/realms/aula/account/`.

Logout errors

Error	Cause	Fix
<code>logout_url</code> is null	<code>sso_id_token</code> not stored (user logged in before migrations ran)	Run <code>php artisan tenants:migrate</code> , then log in fresh

Error	Cause	Fix
"does not support this operation" (broker API 400)	Store Tokens not enabled on the IdP	Enable Store Tokens and Stored Tokens Readable on the IdP in aula realm
"Client not authorized" (broker API 403)	User token lacks broker realm role	Add broker to aula realm default roles; ensure roles scope is in aula-backend default client scopes
"Invalid redirect uri" (IdP logout)	aula realm logout URL not in IdP's client	Add https://sso.aula.de/ to Valid post logout redirect URIs of keycloak-broker in the IdP realm
Backchannel logout loops	Keycloak backchannel to IdP conflicts with browser-based flow	Do not enable backchannel logout — the browser-based chain handles both sessions

Fully delete a test user — users exist in three stores: 1. mock-iserv realm → Users (Keycloak admin) 2. aula realm → Users (Keycloak admin) 3. aula tenant database:

```
docker compose exec aula-backend.v2 php artisan tinker
tenancy()->initialize(\App\Models\Tenant::where('instance_code', 'CODE')->first()
\App\Models\LegacyUser::where('email', 'user@example.com')->delete());
```